

UNIVERSITY OF LUXEMBOURG
Faculty of Science, Technology and Medicine

Profiler-Adaptive Data Staging for Large-Scale Satellite-Image Segmentation on HPC Systems

A Master's Thesis submitted in partial fulfillment
of the requirements for the degree of
Master in Information and Computer Sciences

Nabeel Ahsan

Supervisor: Dr. Leon van Der Torre

Advisor: Richard Albrecht

Reviewer: Dr. Andrea Binsfield

Luxembourg, [MONTH] 2026

Declaration of Authorship

I hereby declare in lieu of an oath that the thesis submitted in partial fulfillment of the requirements for the Master in Information and Computer Sciences and entitled

Profiler-Adaptive Data Staging for Large-Scale Satellite-Image Segmentation on HPC Systems

is original and the result of my own investigations. I have cited in proper and due form all used sources (e.g., ideas, equations, figures, text, tables, programs, algorithms). This work has not been submitted in whole or in part for another degree.

Luxembourg, [MONTH] 2026

Nabeel Ahsan

Acknowledgements

[Write this yourself — it should be in your own voice. Conventionally: supervisor, advisor, reviewer, then the HPC facility, then family.]

The experiments presented in this work were carried out using the HPC facilities of the University of Luxembourg: <http://hpc.uni.lu>

This work builds directly on the dataset and reference implementation released by Casini et al. [3] under an open licence, without which the reproduction reported in Appendix A would not have been possible.

Abstract

Deep learning training on high-performance computing (HPC) clusters is conventionally treated as a problem of accelerator throughput, and optimisation effort is directed at the model. For semantic segmentation over remotely sensed imagery, however, the training corpus is typically stored as very large numbers of small image and mask files on a shared parallel filesystem, and the cost of reaching those files is borne once per sample. We show that for a representative archaeological site-detection workload — the detection of mounded settlement sites (*tells*) in the southern Mesopotamian floodplain [3] — the accelerator is frequently idle while the data path is the binding constraint, and that the loader configuration moves end-to-end segmentation quality by a margin comparable to the entire architectural search reported in the original study. We introduce PADS-Loader (Profiler-Adaptive Data Staging), a drop-in replacement for the PyTorch `DataLoader` that profiles its own behaviour over a short warm-up window, classifies the dominant bottleneck from the measured timings, selects a staging policy — loose files, tar shards, or node-local scratch staging — and executes that policy with a prefetch depth derived from the measurements rather than supplied by the user. Because the sample permutation is fixed at the start of each epoch, the identity of future file accesses is known exactly, so prefetching is exact rather than speculative. We evaluate PADS-Loader against an unmodified loader, a manually tuned loader, fixed tar-shard streaming, and a full pre-staging upper bound, on three segmentation architectures spanning a CNN encoder-decoder, an attention-augmented CNN, and a transformer. **[TO MEASURE: headline throughput and GPU-utilisation result, once the HPC evaluation is complete]** Alongside the positive throughput claim we make a negative accuracy claim: a pipeline optimisation is only sound if segmentation quality is preserved, and we therefore report intersection-over-union against a reference reproduction throughout.

Keywords: Data Staging; I/O Bottlenecks; Semantic Segmentation; Remote Sensing; High-Performance Computing; Archaeological Prospection

Contents

Acknowledgements	3
Abstract	4
Contents	4
List of Figures	8
List of Tables	9
1 Introduction	11
1.1 Background and Motivation	11
1.2 The problem	11
1.3 Research question	12
1.4 Contributions	12
1.5 Thesis structure	13
2 Literature Review	14
2.1 Semantic segmentation architectures	14
2.2 Deep learning for archaeological prospection	15
2.3 Input pipelines and I/O for large-scale training	15
2.4 Summary	16
3 Materials and Datasets	17
3.1 Overview	17
3.2 Dataset: Casini et al. (2023)	17
3.2.1 Provenance and construction	17
3.2.2 Class imbalance	18
3.2.3 Evaluation metrics	19
3.2.4 Reference results	19
3.3 The dataset as an I/O workload	20
3.4 Models	20
3.4.1 MA-Net	21
3.4.2 DeepLabV3+	21
3.4.3 SegFormer	21
3.4.4 Measured characteristics (Bing 1k, default loader)	21
3.5 Hardware and software environment	22

4	Methodology	23
4.1	Motivation: when the DataLoader becomes the bottleneck	23
4.2	PADS-Loader: system overview	23
4.2.1	Stage 1 — Warm-up profiling	24
4.2.2	Stage 2 — Bottleneck classification	24
4.2.3	Stage 3 — Policy selection	25
4.2.4	Stage 4 — Predictive staging	26
4.3	Metrics captured	27
4.4	Evaluation plan	27
4.5	Experimental protocol	28
5	Results	29
5.1	RQ1: When does the DataLoader become the bottleneck?	29
5.2	RQ2: Do tar shards reduce many-small-file overhead?	29
5.3	RQ3: Does predictive staging outperform fixed streaming?	30
5.4	RQ4: Can prefetch depth be auto-tuned?	30
5.5	Profiling overhead	31
5.6	Accuracy is preserved	31
6	Ablation Study	32
6.1	Component ablation	32
6.2	Scratch capacity: the regime the design targets	32
6.3	Input size and the CORONA variant	33
6.4	Classifier threshold sensitivity	33
6.5	Shuffling randomness	33
7	Conclusions	35
7.1	Summary of contributions	35
7.2	Answers to the research questions	36
7.3	What this work does not claim	36
7.4	Closing remarks	36
8	Limitations and Future Work	37
8.1	Limitations	37
8.2	Future work	37
	Bibliography	38
A	Reproduction of the Reference Configuration	41
A.1	Environment	41
A.2	Faithfulness of the port	41
A.3	Result and the batch-size constraint	42
A.4	Metric reduction	42
A.5	Error structure	42
B	Pipeline Defects Identified During Reproduction	44
B.1	Defects affecting the staging mechanism	44
B.1.1	Policy selected after the datasets it selects were constructed	44
B.1.2	No archive the shard reader could open	44
B.1.3	Staging silently degraded to a plain read	45
B.2	Defects affecting segmentation accuracy	45

B.3 Defects affecting reproducibility and operations	46
B.4 Implication for prior results	46

List of Figures

Introduction	11
--------------	----

List of Tables

Materials and Datasets	17
3.1 IoU scores reported by Casini et al. [3] for their experimental configurations.	19
3.2 Baseline characteristics with the default PyTorch <code>DataLoader</code> .	21
3.3 Characteristics with loader parameters tuned (<code>num_workers=4</code> , <code>pin_memory=True</code>).	22
Methodology	23
4.1 Signals recorded during the warm-up window.	24
4.2 Bottleneck classification rules.	25
4.3 The three staging policies.	26
4.4 Evaluation methods and the contrast each provides.	28
Results	29
5.1 Per-stage timing breakdown across architectures and loader configurations. [TO MEASURE: populate from cluster profiling runs]	29
5.2 Filesystem operations issued against GPFS per epoch. [TO MEASURE: populate]	30
5.3 End-to-end comparison. Method 2 is the bar PADS-Loader must clear; method 4 is the ceiling. [TO MEASURE: populate]	30
5.4 Fixed prefetch depths against the auto-tuned depth. [TO MEASURE: populate]	31
5.5 Cost of adaptation. [TO MEASURE: populate]	31
5.6 Segmentation quality under each data path. Reduction convention as Section A.4. [TO MEASURE: populate]	31
Ablation Study	32
6.1 Component ablation. [TO MEASURE: populate]	32
6.2 Behaviour as scratch capacity is constrained. Method 4 of Section 4.4 is unavailable below 100%; PADS-Loader should degrade gracefully rather than fail. [TO MEASURE: populate]	32
6.3 Effect of input configuration. [TO MEASURE: populate]	33
6.4 Sensitivity of the policy decision to the classification threshold. [TO MEASURE: populate]	33
6.5 Trade-off between shuffling fidelity and throughput. [TO MEASURE: populate]	34

<i>LIST OF TABLES</i>	10
Reproduction of the Reference Configuration	41
A.1 Effect of the aggregation convention on reported IoU for an identical model. The reference convention counts a correctly-predicted empty tile as a perfect match.	42
A.2 Per-image IoU by the fraction of the cropped tile occupied by the annotated site.	43
Pipeline Defects Identified During Reproduction	44
B.1 Accuracy-affecting defects.	45

Chapter 1

Introduction

1.1 Background and Motivation

Remote sensing has become a central instrument of archaeological prospection. Where a field survey of a large region once required years of systematic ground inspection, openly available satellite imagery now permits candidate sites to be identified from a desk, and the resulting shortlist to direct where limited excavation effort is spent [3, 15]. The southern Mesopotamian floodplain is a canonical setting for this: it contains thousands of mounded settlement sites, locally called *tells*, formed by the accumulated building levels and occupation debris of communities that remained in one place over long periods [22]. Their colour and shape make them visible from above, and that visibility is what makes automated detection plausible at all.

Semantic segmentation — assigning a class label to every pixel — is well matched to this task, because archaeologists need the outline of a site rather than merely its presence. Casini et al. [3] fine-tuned pretrained segmentation networks on a corpus of roughly five thousand ground-truthed sites and reported intersection-over-union (IoU) in the range of 74–83%, with site-level detection accuracy near 80% once a domain expert had adjudicated the predictions. That work also established the framing that this thesis inherits: the model is a component in a human–AI workflow, not a replacement for the archaeologist.

The present work takes that workload as given and asks a different question. Training such models on a shared HPC cluster is conventionally treated as a problem of accelerator throughput: the model is profiled, the batch size is tuned, mixed precision is enabled, and the data path is assumed to keep up. For this workload that assumption does not hold. The corpus consists of thousands of individually stored image and mask files on a shared parallel filesystem, and each training step issues an independent `open/stat/read/close` sequence per sample against a filesystem optimised for large sequential transfers rather than for many-small-file metadata operations. The accelerator waits.

1.2 The problem

Three properties of this setting, taken together, define the problem addressed here.

The bottleneck is real and it is not the model. Profiling the baseline pipeline across three architectures shows a ratio of data-path time to compute time large enough that the loader is a plausible constraint rather than a rounding error, and the effect is most pronounced for the architecture with the lowest per-step compute — precisely the configuration in which a fast model exposes a slow pipeline.

The bottleneck moves, but the configuration does not. `num_workers`, `prefetch_factor` and `pin_memory` are fixed at loader construction and never revisited. On a shared filesystem, observed read latency is not a property of the dataset alone but of whatever else is running on the cluster, and therefore varies over the lifetime of a single job. A statically configured loader cannot respond to a bottleneck that shifts underneath it.

Future accesses are known exactly, not merely predictable. Once the epoch permutation is drawn, the sequence of samples — and hence the sequence of files — is fully determined. Prefetching in this setting need not be speculative. This is the property that the method exploits, and it is what separates an exact staging schedule from a heuristic cache.

1.3 Research question

The thesis addresses one question:

Can a profiler-adaptive **DataLoader** that combines bottleneck classification, tar-shard staging, and predictive prefetching improve GPU utilisation and tile throughput for large-scale satellite-image segmentation on HPC systems?

which decomposes into four sub-questions, each answered in this work by a specific experimental contrast rather than by an aggregate result:

RQ1. When does the **DataLoader** become the bottleneck on a shared parallel filesystem?

RQ2. Do tar shards reduce many-small-file overhead?

RQ3. Does predictive scratch staging outperform fixed shard streaming?

RQ4. Can prefetch depth be auto-tuned from measured timings?

1.4 Contributions

1. **A characterisation of the workload as an I/O problem.** We describe the segmentation corpus not only by its class balance and evaluation metrics, as is conventional, but by its file count, size distribution and access pattern — the characterisation that motivates staging and that the segmentation literature usually leaves implicit (Chapter 3).
2. **PADS-Loader, a profiler-adaptive data-staging loader.** A drop-in **DataLoader** replacement that measures T_{data} , T_{decode} , T_{h2d} and T_{gpu} over a bounded warm-up window, classifies the dominant bottleneck by an inspectable rule set, selects among loose, shard and stage policies, and derives its prefetch depth from the measured staging and consumption times rather than from a user-supplied constant (Chapter 4).
3. **An evaluation that isolates each mechanism.** PADS-Loader is compared against an unmodified loader, a manually tuned loader, fixed tar-shard streaming, and a full pre-staging upper bound, so that the benefit of sharding is separated from the benefit of staging, and the benefit of staging from the benefit of *predicting* what to stage (Chapter 5).

4. **A reproduction of the reference segmentation results, and an account of what it takes to obtain them.** We reproduce the reference configuration of Casini et al. [3] and document the pipeline defects and environmental sensitivities encountered in doing so, including a case in which reduced batch size — forced by accelerator memory — degrades IoU substantially through its effect on batch normalisation statistics (Appendix A).

1.5 Thesis structure

Chapter 2 reviews segmentation architectures, deep learning for archaeological prospection, and prior work on I/O for large-scale training. Chapter 3 describes the dataset in both of its characterisations and the three model architectures. Chapter 4 presents PADS-Loader. Chapter 5 reports the evaluation, and Chapter 6 isolates the contribution of individual components. Chapter 7 concludes, and Chapter 8 discusses limitations and further work.

Chapter 2

Literature Review

This chapter reviews three bodies of work that the thesis sits between: semantic segmentation architectures (Section 2.1), deep learning applied to archaeological prospection (Section 2.2), and input-pipeline performance for large-scale training (Section 2.3). The first two establish the workload; the third establishes the gap that PADS-Loader addresses.

2.1 Semantic segmentation architectures

Semantic segmentation assigns a class label to every pixel of an image. The modern formulation begins with fully convolutional encoder–decoder networks, of which U-Net [17] is the canonical instance: a contracting path captures context, an expanding path recovers spatial resolution, and skip connections carry high-frequency detail across the bottleneck. U-Net was introduced for biomedical imaging under severe data scarcity, which is one reason it transferred well to remote sensing, where annotated corpora are similarly small.

Two families extend this design in directions relevant here. The first enlarges the effective receptive field without sacrificing resolution. DeepLabV3+ [4] combines atrous (dilated) convolution and atrous spatial pyramid pooling with a decoder module that recovers object boundaries, capturing multi-scale context at high capacity. The second inserts attention into the encoder–decoder. MA-Net [6] augments a U-Net-like structure with self-attention in the latent space, allowing the network to weight latent features by content — an idea imported from the transformer literature [21]. Developed for liver and tumour segmentation, MA-Net has since been applied to remote sensing, and it is the architecture selected as best-performing by Casini et al. [3], which is why it is the reproduction baseline here.

A third family replaces the convolutional encoder entirely. SegFormer [24] pairs a hierarchical, positional-encoding-free transformer encoder with a lightweight all-MLP decoder, achieving competitive accuracy at markedly lower per-step compute than heavy CNN decoders. For the purposes of this thesis that low compute cost is not incidental: it makes SegFormer the configuration in which data-loading stalls are most exposed, since a model that computes quickly spends proportionally more of its time waiting.

Encoder choice is largely orthogonal to decoder architecture. EfficientNet [18] obtains strong accuracy-per-parameter through compound scaling of depth, width and resolution, and EfficientNet-B3 is used throughout this work as the encoder for both CNN decoders, following the reference study. All architectures are instantiated through `segmentation_models_pytorch` [9], which allows encoder and decoder to be selected independently and supplies the loss functions used here.

2.2 Deep learning for archaeological prospection

Archaeology is what Bickler terms a “slow data” field: annotated examples accrue through fieldwork, not through scraping, and corpora are correspondingly small. The standard response is transfer learning — initialising from a model pretrained on a large general corpus such as ImageNet [5] and fine-tuning on the specific task.

Applications divide by output granularity. Tile classification labels a map tile as containing a site or not; object detection predicts bounding boxes; semantic segmentation predicts a shape. Orengo et al. [15] applied machine-learning classification to multisensor and multitemporal satellite data for mound detection, working at the level of per-pixel classification over large regions. Trier et al. [19] used deep networks on airborne laser scanning data for semi-automatic mapping of archaeological topography, and Guyot et al. [8] combined detection and segmentation of archaeological structures from LiDAR. Verschoof-van der Vaart and Lambers [20] report on applying automated object detection within archaeological practice, emphasising the integration question rather than the accuracy question alone.

Two themes from this literature bear directly on the present work. First, evaluation is unusually fraught. Fiorucci et al. [7] argue that standard detection measures are poorly matched to archaeological objects on LiDAR and propose alternatives; Casini et al. [3] make the related observation that IoU and site-level detection accuracy can move in opposite directions, because a more conservative model produces fewer positive predictions, shrinking the union term and raising IoU while missing sites and lowering recall. Any model comparison that relies on IoU alone must be read with that caveat.

Second, labels are noisy in a structured way. A substantial fraction of catalogued sites are no longer visible on modern imagery, having been levelled for agriculture or obscured by development; a model that correctly produces no contour for such a tile is penalised as a false negative against the annotation. Casini et al. [3] address this with human-in-the-loop adjudication, which raises reported detection accuracy from roughly 63% to 80%. This is a property of the corpus, not of any particular model, and it bounds what any reproduction can be expected to achieve.

2.3 Input pipelines and I/O for large-scale training

The literature above is concerned with what the model computes. A separate and smaller literature is concerned with whether the accelerator is given anything to compute.

Mohan et al. [12] characterise *data stalls* in DNN training and show that they are common rather than exceptional: across a range of workloads, accelerators spend a substantial fraction of wall-clock time blocked on fetching and preprocessing, and the effect worsens as accelerators get faster relative to storage and host CPUs. Their system, CoordDL, addresses this through coordinated caching and by eliminating redundant preprocessing across concurrent jobs. Murray et al. [13] describe `tf.data` and make the same argument from the framework side, arguing that input pipelines deserve to be treated as first-class, optimisable programs rather than as glue code.

The many-small-file problem is a specific and well-known pathology on parallel filesystems. Metadata cost per file is close to constant regardless of how few bytes are read, so per-file overhead does not amortise, and a corpus of thousands of small images generates metadata traffic out of proportion to the data actually transferred. The standard mitigation is to aggregate samples into large sequentially read archives. WebDataset [1] formalises this: samples are stored in POSIX tar archives and streamed sequentially, converting many small random reads into few large sequential ones, at the cost of exact

random-access shuffling — which is approximated by shuffling shard order and maintaining an in-memory sample buffer. NVIDIA DALI [14] attacks an adjacent part of the problem by moving decode and augmentation onto the GPU.

Two gaps in this body of work motivate PADS-Loader.

First, these systems are *statically configured*. WebDataset streams shards at a fixed depth; the number of loader workers and the prefetch factor are chosen once. None of them observes whether the configuration is in fact sufficient to hide the transfer, and none revises it when contention on a shared filesystem changes the answer mid-run.

Second, and more specifically, they do not exploit the fact that in supervised training with a fixed epoch permutation, *future accesses are known exactly*. Caching and prefetching are ordinarily speculative because future references must be predicted. Here they need not be: once the permutation is drawn, the identity of the next k samples is determined, and a staging schedule can be computed rather than guessed. PADS-Loader is built on that observation.

2.4 Summary

Segmentation architecture for this workload is a solved problem in the sense that matters here: the reference study found that architectural choice moves IoU by only a few percentage points, plausibly within the fluctuation induced by random augmentation [3]. If the remaining headroom is not in the model, it may be in the pipeline that feeds it. The I/O literature establishes that data stalls are real and that aggregation into shards mitigates the dominant filesystem pathology, but leaves the configuration static and the prefetch speculative. This thesis addresses that gap.

Chapter 3

Materials and Datasets

3.1 Overview

This chapter describes the data and the model architectures used throughout the thesis. The experimental subject is a semantic segmentation workload over satellite imagery of the southern Mesopotamian floodplain, and the object of study is the data pipeline that feeds that workload on a shared HPC filesystem. The dataset is therefore treated in two distinct ways: as a machine learning dataset, characterised by its imagery, class balance and evaluation metrics (Section 3.2); and as an I/O workload, characterised by file count, file size distribution and access pattern (Section 3.3). The second characterisation is the one that motivates PADS-Loader, and it is the one usually left implicit in the segmentation literature.

3.2 Dataset: Casini et al. (2023)

The main experiments use the dataset introduced by Casini et al. [3], which addresses semantic segmentation and detection of mounded archaeological sites — tells — across the southern Mesopotamian floodplain in Iraq.

3.2.1 Provenance and construction

The dataset originates from a corpus of geo-referenced vector shapes corresponding to the contours of known mound sites within the survey area of the Floodplains Project, spanning approximately 66,000 km². The vector corpus was assembled at the University of Bologna by consolidating all published archaeological surveys in the region and geo-referencing the sites catalogued therein; it contains 4,934 shapes, each corresponding to a site confirmed by ground-truthing and by study of the associated surface scatter of artefacts.

Because the corpus was compiled as an archaeological reference rather than as a training set, the authors applied two filtering passes before image generation:

1. **Removal of the 200 largest sites by area.** These exceed the area of the square input region and would therefore yield an almost entirely positive segmentation mask, providing no usable learning signal. Visual inspection confirmed that they follow the shape of extended areas rather than of simple mounds.
2. **Removal of 684 sites** that were either too small to be a tell or marked as destroyed. The size threshold was set at roughly 1,000 m², corresponding to a circle of 30 m diameter. Sites below this threshold typically correspond to generic annotations for known sites of unknown size or imprecise location.

Image generation was performed in QGIS. For each retained site, a Python script exported a square of side L centred on the site centroid containing only Bing Maps satellite imagery, accessed through the QuickMapService plugin. The same square was exported a second time without the base map but with the site contour rendered as a solid filled shape, giving the corresponding binary ground-truth mask. The task is therefore binary: one class for mounded sites, one for everything else.

Two spatial configurations were produced: $L = 1000$ m (“1k”) and $L = 2000$ m (“2k”). Imagery was exported at approximately 1 pixel per metre — 1024 px for 1000 m, and double that for the larger extent — and subsequently downsampled by a factor of two to reduce computational cost, which the original authors report as having low impact on accuracy. At training time a square of side $L/2$ is randomly cropped from the exported tile; this both prevents the model from learning a positional prior in which sites always appear at the image centre, and acts as data augmentation. Further augmentation consists of random rotation, mirroring, and small shifts in brightness and contrast, resampled at each training iteration.

Finally, 1,155 images with empty masks — containing no site to predict — were added, sampled from locations proposed by the archaeologists: highly urbanised areas, intensive agriculture, flood-prone terrain such as artificial lakes and basins, and rocky hills and mountains. These negatives are what force the model to learn what a tell is *not*. The final dataset consists of 5,025 image/mask pairs, split 90% training / 10% held-out test, stratified over the empty images, with a further 10% of the training partition randomly reserved as a validation set.

A CORONA-augmented variant also exists, in which declassified historical satellite imagery is imported into QGIS and cropped identically to the Bing tiles, giving a second input channel group. Because CORONA was used only as a complement in the original work, sites destroyed after the 1970s were excluded from that analysis.

Dataset variant used in this thesis. All experiments reported here use the Bing 1k configuration unless otherwise stated. This choice is deliberate: the 1k configuration is the smallest of the published variants and therefore the least favourable to a data-staging argument — a pipeline that is I/O-bound on 1k tiles will be at least as I/O-bound on 2k tiles or on the CORONA-augmented input. Chapter 6 revisits the larger input configurations.

3.2.2 Class imbalance

The segmentation target is severely imbalanced: mound pixels constitute a small minority of any given tile, and 1,155 of the 5,025 tiles contain no positive pixels at all. Direct measurement over the corpus used here gives a mean positive-pixel fraction of approximately 9.6% per site-bearing tile, with a median near 7%. The masks themselves are cleanly binary — 90.4% of mask pixels take the background value and 8.8% the foreground value, with only 0.7% at intermediate values — so label ambiguity at object boundaries is not a material source of error.

This imbalance is the reason the original work adopts focal loss [11] over Dice loss, and the reason the primary reported metric is IoU rather than pixel accuracy. It is also, importantly for this thesis, the reason the workload is decode-heavy relative to its useful signal: a substantial fraction of the bytes read from the filesystem contribute nothing but negative pixels.

3.2.3 Evaluation metrics

The original paper evaluates using intersection-over-union at the pixel level, and accuracy, precision, recall and the Matthews correlation coefficient at the site-detection level:

$$\text{IoU} = \frac{|P \cap G|}{|P \cup G|} \quad (3.1)$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (3.2)$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP} \quad (3.3)$$

$$\text{MCC} = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (3.4)$$

where P denotes the predicted shape and G the ground-truth shape.

The distinction between the two levels matters. IoU measures how well a predicted contour overlaps an annotated one; it does not measure how many sites were found. The original authors note that IoU can improve while detection accuracy degrades, as occurs between their Model 5 and Model 6: a more conservative model produces fewer positive predictions, shrinking the union term and raising IoU, while missing sites and lowering recall.

Reduction convention. A subtlety that materially affects reported numbers is how per-image scores are aggregated. The reference implementation reports `smp.metrics.iou_score(..., reduction="macro-imagewise")`: IoU is computed per image and then averaged, with the library's convention that an empty ground truth correctly predicted empty scores 1.0. This differs from a globally pooled IoU, in which true positives, false positives and false negatives are summed across the whole set before the ratio is taken. On the same trained model the two conventions differ by several percentage points, with the per-image reduction the higher of the two because the empty tiles contribute perfect scores. All comparisons in this thesis therefore state which reduction is used. Section A.4 quantifies the difference.

3.2.4 Reference results

Casini et al. report the IoU scores in Table 3.1 on the held-out test set. Standard deviations arise from repeated testing to average out the random crop, which is applied at test time as well as during training.

Model	Input	Filter	IoU (%)	Std. dev
Model 1	Bing 1k		74.17	0.38
Model 2	Bing 1k	✓	78.10	0.54
Model 3	Bing + CORONA 1k		74.06	0.39
Model 4	Bing 2k		79.77	0.34
Model 5	Bing 2k	✓	81.54	0.35
Model 6	Bing + CORONA 2k	✓	83.45	0.18

Table 3.1: IoU scores reported by Casini et al. [3] for their experimental configurations.

At the detection level, Model 5 achieves 62.57% accuracy under automatic evaluation and 80.40% after human-in-the-loop adjustment for mislabelled (no-longer-visible) sites;

Model 6 achieves 60.08% and 79.13% respectively. Their configuration search compared U-Net against MA-Net, ResNet-18 against EfficientNet-B3, and Dice loss against focal loss, settling on MA-Net + EfficientNet-B3 + focal loss trained for 20 epochs as the best configuration; they observe that the differences between configurations were small, within a few percentage points, and plausibly attributable to fluctuation from the random augmentation.

These figures serve two purposes here. First, they establish an accuracy floor: any optimisation to the data pipeline proposed in this thesis must be shown not to degrade segmentation quality relative to this reference. Second, the observation that architectural choice moves IoU by only a few points motivates the framing of this work — if the model is not where the remaining headroom is, the pipeline may be.

3.3 The dataset as an I/O workload

The above is the standard description of the dataset. For the purposes of this thesis a second description is required, because the property of the dataset that PADS-Loader responds to is not its class balance but its file layout.

The Bing 1k dataset consists of 5,025 image files and 5,025 corresponding mask files stored as individual objects on the shared GPFS filesystem. Each training step therefore issues, per sample, an independent `open/stat/read/close` sequence against a parallel filesystem that is optimised for large sequential transfers rather than for many-small-file metadata operations. The relevant properties are:

Many small files. Metadata cost per sample is close to constant regardless of how few bytes are actually read, so per-file overhead does not amortise.

Paired access. Every image read is accompanied by a mask read at a different path, doubling the metadata traffic per sample.

Shared contention. GPFS is a shared resource; observed read latency is not a property of the dataset alone but of concurrent jobs on the cluster, and is therefore non-stationary across a run.

Deterministic access order. Once the epoch permutation is fixed, the sequence of samples — and hence of files — is known in advance. Future data access is predictable, which is precisely the precondition that makes proactive prefetching and shard staging exact rather than speculative.

The final point is the hinge of the entire method. The dataset is small enough to fit on node-local scratch and its access pattern is known ahead of time; the fact that the baseline pipeline nonetheless leaves the GPU idle is a property of the loader, not of the data.

3.4 Models

Three segmentation architectures are used, spanning a CNN encoder–decoder, an attention-augmented CNN, and a transformer. All three were trained under an identical data pipeline so that pipeline effects can be isolated from architectural effects.

3.4.1 MA-Net

MA-Net (Multi-scale Attention Network) [6] extends the U-Net encoder–decoder with a self-attention mechanism inserted in the latent space, allowing the network to weight latent features according to content. Originally developed for liver and tumour segmentation in medical imaging, it has since been applied to remote sensing. It is included here because it is the architecture selected as best-performing by Casini et al., and therefore constitutes the reproduction baseline for this thesis. Implementation: `segmentation_models_pytorch` [9] with an EfficientNet-B3 [18] encoder pretrained on ImageNet [5].

3.4.2 DeepLabV3+

DeepLabV3+ [4] uses atrous (dilated) convolution and atrous spatial pyramid pooling to capture multi-scale context without loss of spatial resolution, combined with a decoder module that recovers object boundaries. It is included as a widely used, high-capacity CNN baseline and represents the heaviest of the three in both parameter count and per-step GPU time. Implementation: `segmentation_models_pytorch` with a ResNet-50 encoder.

3.4.3 SegFormer

SegFormer [24] pairs a hierarchical, positional-encoding-free transformer encoder with a lightweight all-MLP decoder. It is included as the transformer baseline and, critically for this thesis, as the least GPU-bound of the three architectures: its low per-step compute makes it the configuration in which data-loading stalls are most exposed. Implementation: the NVlabs reference architecture as distributed through the HuggingFace `transformers` library.

3.4.4 Measured characteristics (Bing 1k, default loader)

Table 3.2 reports measurements on the Iris cluster on a single V100 with the baseline PyTorch `DataLoader` at default settings, and Table 3.3 the same with loader parameters tuned (`num_workers=4`, `pin_memory=True`).

	SegFormer	MA-Net	DeepLabV3+
Trainable parameters	3.7 M	17.2 M	26.7 M
Model size (MB)	15	68	106
IoU (%)	67	68	35.9
Epochs	34	21	46
Batch size	32	32	32
CPU time (s)	3.922	4.512	4.639
GPU time (ms)	416.305	644.360	830.446

Table 3.2: Baseline characteristics with the default PyTorch `DataLoader`.

Two observations from these tables set up Chapter 4. First, the ratio of CPU time to GPU time per step is large enough across all three architectures that the data path is a plausible bottleneck rather than a rounding error. Second, merely changing two loader flags moves MA-Net IoU by five points and DeepLabV3+ by seven — a swing comparable in magnitude to the entire architectural search in the original paper. That the loader

	SegFormer	MA-Net	DeepLabV3+
Trainable parameters	3.7 M	17.2 M	26.7 M
Model size (MB)	15	68	106
IoU (%)	67	73	43
Epochs	34	22	43
Batch size	32	32	32
CPU time (s)	3.922	4.737	5.245
GPU time (ms)	416.305	623.832	910.236

Table 3.3: Characteristics with loader parameters tuned (`num_workers=4`, `pin_memory=True`).

configuration is competitive with the architecture choice as a source of variation is the empirical hook for this work.

3.5 Hardware and software environment

All primary experiments were run on the Iris cluster at the University of Luxembourg, on GPU nodes equipped with NVIDIA V100 accelerators, scheduled via Slurm. Storage is provided by a shared GPFS filesystem, with node-local scratch available for staging. The software stack is containerised with Apptainer to guarantee a reproducible environment across nodes; the container image bundles PyTorch [16], PyTorch Lightning, `segmentation_models_pytorch` [9] and `albugmentations` [2].

A secondary, single-GPU workstation environment (NVIDIA RTX 3050 Laptop, 4 GB) was used for development, defect analysis and reproduction work. Appendix A documents that environment and, importantly, the respects in which results obtained on it are *not* comparable to the cluster results reported above — most consequentially the accelerator memory limit, which forbids the batch size of 32 used throughout the primary experiments and thereby degrades batch-normalisation statistics. That constraint is the reason all accuracy figures in this thesis are drawn from the cluster.

Chapter 4

Methodology

4.1 Motivation: when the DataLoader becomes the bottleneck

Deep learning training on HPC is conventionally treated as a GPU problem. The profiling results reported in Section 3.4.4, together with a targeted profiling pass over the baseline pipeline, indicate that for this workload the GPU is frequently not the constraint. Four findings motivate PADS-Loader:

1. **GPU starvation.** With `num_workers=0`, the accelerator sits idle between batches. It is waiting on the data path, not on the model.
2. **Shared filesystem pressure.** Thousands of small image and mask files generate high metadata overhead on GPFS. The cost is dominated not by bytes transferred but by per-file operations against a shared parallel filesystem.
3. **Static configuration.** `num_workers`, `prefetch_factor` and `pin_memory` are set once at loader construction and never adapt. They cannot respond to a bottleneck that shifts — as it does, when contention on a shared filesystem varies over the lifetime of a job.
4. **Deterministic access.** Because the sample order is fixed once the epoch permutation is drawn, the sequence of future file accesses is known. Prefetching is therefore not speculative: it can be exact.

4.2 PADS-Loader: system overview

PADS-Loader (Profiler-Adaptive Data Staging) is a drop-in replacement for the PyTorch `DataLoader` that observes its own behaviour during a short warm-up window, classifies the dominant bottleneck from the observed timings, selects a staging policy accordingly, and then executes that policy with predictive prefetching for the remainder of the run.

The pipeline has four stages:

1. **Warm-up profiling** — 50–100 batches, instrumented.
2. **Bottleneck classification** — I/O, decode, H2D or compute.
3. **Policy selection** — loose, shard or stage.
4. **Predictive staging** — shard prefetch to node-local scratch.

The design intent is that stages 1–3 cost a bounded, one-off overhead — tens of seconds on a job measured in hours — and that stage 4 recovers that cost many times over. The loader is deliberately conservative by default: if no bottleneck is detected, it degrades to the loose policy, which is the ordinary PyTorch `DataLoader`.

4.2.1 Stage 1 — Warm-up profiling

For the first 50–100 batches, PADS-Loader instruments the data path and records per-batch timings without altering behaviour. The signals measured are given in Table 4.1.

Signal	Meaning
T_{data}	Time spent waiting for the next batch to become available
T_{decode}	Image and mask decode time, plus augmentation transforms
T_{h2d}	Host-to-device (CPU→GPU) transfer time
T_{gpu}	Model forward + backward time
GPU utilisation	Accelerator occupancy over the window
Queue fullness	<code>DataLoader</code> output-queue depth
Shard size	Whether the next shard can be staged efficiently
Scratch availability	Whether node-local scratch has free capacity

Table 4.1: Signals recorded during the warm-up window.

The final two are feasibility signals rather than performance signals: they determine whether a policy is *available*, not whether it is desirable.

Instrumentation. The four timers are implemented as follows. T_{gpu} and T_{h2d} are measured with `torch.cuda.Event` pairs recorded around the transfer and the forward/backward call respectively, synchronised before reading, because wall-clock timing of asynchronous CUDA work would otherwise attribute queueing delay to the wrong stage. T_{data} is measured as wall-clock time around `next(iterator)`, which captures the time the training loop is blocked waiting for a batch — and is therefore the signal that directly expresses starvation. T_{decode} is instrumented inside `__getitem__`, accumulating per-sample decode and transform time, and is aggregated per batch.

Queue fullness is sampled rather than integrated: a persistently full output queue means the consumer is the constraint and the loader is keeping up; a persistently empty one means the reverse. It therefore disambiguates the case where T_{data} is low simply because the model is slow.

Aggregation. Statistics are taken as the median over the window rather than the mean. The first batches of a run include CUDA context creation, cuDNN algorithm selection and worker process start-up, all of which are one-off costs that would inflate a mean and bias the classifier towards diagnosing an I/O bottleneck that does not exist in steady state. The first $w_0 = 10$ batches are discarded before statistics are computed, and the median is taken over the remainder.

4.2.2 Stage 2 — Bottleneck classification

The profiled signals are mapped onto a dominant bottleneck class. The classification is not a black box: it is a small set of rules over the measured timings, chosen so that the decision is inspectable and reproducible.

Detected condition	Interpretation	Action
T_{data} low, T_{gpu} high, GPU utilisation high	Compute-bound; the pipeline is already keeping up	Loose
Many <code>open/stat/read</code> calls; high metadata overhead	Many-small-file overhead dominates	Shard
Shared filesystem read latency high; scratch available	Shared-FS latency dominates and can be avoided	Stage
Many-small-file overhead <i>and</i> slow shared FS	Both pathologies present	Shard + Stage
T_{decode} dominates	CPU-side decode is the constraint	Increase workers / cache decoded samples

Table 4.2: Bottleneck classification rules.

Policies are not mutually exclusive. Scratch staging and an increased worker count may both be active simultaneously when filesystem I/O and decode are co-bottlenecks. The classifier therefore emits a *set* of actions rather than a single label.

Thresholds. The rules are expressed as ratios rather than absolute times, so that they transfer across accelerators of different speeds. Writing $\tau = \max(T_{\text{decode}}, T_{\text{gpu}})$, the implementation used here declares:

$$\text{stage} \quad \text{if} \quad \text{scratch available} \wedge T_{\text{h2d}} > \gamma_s \cdot \max(\tau, \epsilon) \quad (4.1)$$

$$\text{shard} \quad \text{if} \quad T_{\text{decode}} > \gamma_d \cdot \max(T_{\text{h2d}}, T_{\text{gpu}}) \vee T_{\text{data}} > \gamma_d \cdot \tau \quad (4.2)$$

$$\text{loose} \quad \text{otherwise} \quad (4.3)$$

with $\gamma_s = 1.25$, $\gamma_d = 1.5$ and a floor ϵ preventing division by near-zero timings on fast local storage. The asymmetry is deliberate: the thresholds are set above unity so that a policy is adopted only when the corresponding cost *dominates*, not merely when it is present. A loader that switches policy on a marginal signal would incur staging cost for no benefit, and the conservative default is the ordinary loader.

A note on what the classifier optimises. The rules above ask “is the data path the bottleneck?”, not “which reader is fastest?”. These are different questions, and the distinction matters when interpreting results. If T_{data} is fully hidden behind T_{gpu} by worker overlap, a reader that is intrinsically faster buys no wall-clock improvement, and the classifier correctly declines to switch. Section 6.4 examines the sensitivity of the outcome to γ_s and γ_d .

4.2.3 Stage 3 — Policy selection

Three staging policies are defined, summarised in Table 4.3.

Loose is the baseline and the fallback. Shard attacks metadata overhead by converting thousands of `open` calls into a handful of large sequential reads. Stage attacks latency

Policy	What it does	Best when
Loose	Reads individual image and mask files directly from GPFS	Simple baseline; small or node-local dataset; workload already compute-bound
Shard	Reads samples from tar archives, amortising per-file metadata cost over many samples	Many-small-file overhead is the problem
Stage	Copies future shards to node-local scratch ahead of use, removing the shared filesystem from the critical path	Shared filesystem latency is the problem

Table 4.3: The three staging policies.

and contention by moving data off the shared filesystem entirely, exploiting the fact that the dataset fits in node-local scratch. The two compose: shards may be both formed and staged.

Archive construction. The shard policy requires archives whose member layout the reader can address by original filename. Archives are built once, offline, as uncompressed POSIX tar files with members named `images/<name>.jpg` and `masks/<name>.png`. Compression is deliberately not applied: the payload is already-compressed JPEG and PNG, so entropy coding yields negligible size reduction while forcing the archive to be decompressed from the stream start on every random access — precisely the access pattern a shuffled loader produces. This is a departure from WebDataset [1], which streams shards sequentially and therefore does not pay that penalty; the choice here preserves exact random access at the cost of forgoing compression.

4.2.4 Stage 4 — Predictive staging

Because the epoch permutation is fixed at the start of the epoch, the identity of the next k shards is known exactly. PADS-Loader exploits this by maintaining a background staging thread that copies shard $i+d$ to node-local scratch while shard i is being consumed, where d is the prefetch depth.

The prefetch depth d is not a hyperparameter supplied by the user but is derived from the measured timings. Let t_{stage} be the measured time to copy one shard to scratch and t_{consume} the measured time to consume one shard’s worth of samples. The depth is set to

$$d = \min\left(\left\lceil \frac{t_{\text{stage}}}{t_{\text{consume}}} \right\rceil + 1, d_{\text{max}}\right), \quad d_{\text{max}} = \left\lfloor \frac{C_{\text{scratch}}}{s_{\text{shard}}} \right\rfloor \quad (4.4)$$

where C_{scratch} is the free capacity of node-local scratch and s_{shard} the shard size. The first term ensures the transfer is covered by the consumption of the shards ahead of it; the $+1$ provides one shard of slack against jitter in t_{stage} , which on a shared filesystem is non-stationary. The second term is the feasibility constraint: no more than d_{max} shards may be resident at once.

This is the mechanism by which RQ4 — can prefetch depth be auto-tuned from measured timings? — is answered affirmatively by construction, and it is what distinguishes PADS-Loader from WebDataset, which streams shards at a fixed, statically configured depth without reference to whether that depth is sufficient to hide the transfer.

Eviction. Staged shards are evicted in consumption order once fully consumed, maintaining at most d resident shards. Because consumption order is known, eviction is exact and requires no recency heuristic.

4.3 Metrics captured

Two families of metric are reported.

Pipeline metrics (the object of the optimisation).

- **GPU utilisation** — mean accelerator occupancy over the run, the headline indicator of whether starvation has been eliminated.
- **Tile throughput** — samples processed per second; the end-to-end figure of merit.
- **Time-to-epoch** — wall-clock seconds per epoch, and total time-to-target-IoU.
- $T_{\text{data}} / T_{\text{decode}} / T_{\text{h2d}} / T_{\text{gpu}}$ **breakdown** — retained for the full run, not just warm-up, so that the *shift* in the bottleneck under each policy is visible rather than merely its disappearance.
- **Filesystem operation counts** — `open/stat/read` calls issued against GPFS, the direct measure of whether sharding achieved what it was intended to achieve (RQ2).
- **Scratch traffic and residency** — bytes staged, staging hit rate, peak scratch occupancy.
- **Profiling overhead** — the cost of the warm-up window itself, reported explicitly so that the net benefit is honest.

Model metrics (the constraint, not the objective).

- IoU on the held-out test set, against the Section 3.2 reference, stating the reduction convention used.
- Epochs to convergence, and detection-level accuracy, precision, recall and MCC where a comparison to Casini et al. is being made.

The reason for reporting both is that this thesis makes a negative claim about accuracy as well as a positive claim about throughput. PADS-Loader is only useful if it accelerates the pipeline *without* degrading segmentation quality. A throughput improvement obtained at the cost of reduced shuffling randomness, and therefore of a worse model, is not an improvement. Any observed IoU deviation greater than the ± 0.5 percentage point standard deviation reported in the reference results must be treated as a regression to be explained, not as noise.

4.4 Evaluation plan

PADS-Loader is compared against four alternatives, all on identical model, dataset and hardware, with only the data path varying (Table 4.4).

The comparison set is constructed so that each sub-question is answered by a specific contrast rather than by the aggregate result:

#	Method	What it isolates
1	Baseline PyTorch DataLoader — loose files, default settings	The unmodified baseline
2	Tuned PyTorch DataLoader — best num_workers, pin_memory, prefetch_factor	How much is achieved by tuning alone; Isolates the benefit of this, not merely the benefit of GPFS (RQ3)
3	WebDataset — fixed tar-shard streaming, no adaptation	Isolates the benefit of the WebDataset API
4	Scratch baseline — loose files copied manually to node-local scratch up front	Isolates the benefit of pre-copying files to the local disk; the ceiling for GPFS from this point forward
5	PADS-Loader (proposed) — adaptive staging + predictive prefetch	—

Table 4.4: Evaluation methods and the contrast each provides.

- **RQ1** is answered by the profiling breakdown across methods 1 and 2 and across the three architectures — SegFormer, with the lowest T_{gpu} , is the configuration in which the loader bottleneck should appear first.
- **RQ2** is answered by 3 vs 1, and by the filesystem operation counts.
- **RQ3** is answered by 5 vs 3, with 4 as the upper bound.
- **RQ4** is answered by 5 vs 3, and by a sweep of fixed depths in the ablation of Chapter 6 — the auto-tuned depth should land at or near the best fixed depth, without having been told it.

Method 4 deserves emphasis: it is the honest ceiling. If manually pre-staging the entire dataset to scratch matches PADS-Loader, then adaptivity has bought nothing for this dataset, and the contribution reduces to the observation that the dataset fits in scratch. PADS-Loader’s claim is that it reaches that ceiling without requiring the user to know in advance that scratch is the answer, and that it continues to work when the dataset does *not* fit — which is the regime the design is actually aimed at, and which Chapter 6 probes by synthetically constraining scratch capacity.

4.5 Experimental protocol

To ensure that pipeline differences are not confounded with training noise, all comparisons hold the following fixed: random seed, epoch permutation, model architecture and initialisation, batch size, optimiser and learning rate, and the augmentation pipeline. Each configuration is repeated **[TO MEASURE: number of repeats]** times and reported as mean $\pm$ standard deviation, because measurements on a shared filesystem are non-stationary and a single run is not evidence. Jobs are submitted through Slurm with identical resource requests, and where possible scheduled on the same node class to control for hardware variation.

Chapter 5

Results

This chapter reports the evaluation described in Section 4.4. Section 5.1 establishes when the loader is the bottleneck (RQ1); Sections 5.2–5.4 address sharding, predictive staging and prefetch-depth auto-tuning in turn. Section 5.6 verifies the accuracy constraint.

All figures are reported as mean $\pm$ standard deviation over [TO MEASURE: n] repeats, since measurements on a shared filesystem are non-stationary and a single run is not evidence.

5.1 RQ1: When does the DataLoader become the bottleneck?

Architecture	Loader	T_{data}	T_{decode}	T_{h2d}	T_{gpu}	GPU util. (%)
SegFormer	default					
SegFormer	tuned					
MA-Net	default					
MA-Net	tuned					
DeepLabV3+	default					
DeepLabV3+	tuned					

Table 5.1: Per-stage timing breakdown across architectures and loader configurations. [TO MEASURE: populate from cluster profiling runs]

The prediction under test is directional and stated in advance: SegFormer, having the lowest T_{gpu} of the three architectures (Table 3.2), should be the configuration in which the loader bottleneck appears first, because a model that computes quickly spends proportionally more of its step waiting. DeepLabV3+, the heaviest, should be the configuration in which staging buys least. If the ordering does not hold, the framing of this thesis is wrong and should be reported as such.

[TO MEASURE: narrative: does the predicted ordering hold?]

5.2 RQ2: Do tar shards reduce many-small-file overhead?

The direct measure is filesystem operation counts, not wall-clock: sharding is intended to convert many small metadata operations into few large sequential reads, and that intent is verifiable independently of whether it happens to pay off in this configuration.

[TO MEASURE: narrative: operation-count reduction, and whether it translates into wall-clock benefit]

Method	open calls	stat calls	Bytes read
1. Baseline loose			
3. WebDataset (shards)			
5. PADS-Loader			

Table 5.2: Filesystem operations issued against GPFS per epoch. **[TO MEASURE: populate]**

A negative result here is publishable and should be reported as such: if operation counts fall substantially but throughput does not, the conclusion is that metadata overhead was not the binding constraint on this system, which is itself an answer to RQ1.

5.3 RQ3: Does predictive staging outperform fixed streaming?

Method	Throughput (samples/s)	GPU util. (%)	s/epoch	IoU	Δ vs. ref.
1. Baseline DataLoader					
2. Tuned DataLoader					
3. WebDataset					
4. Full pre-stage (ceiling)					
5. PADS-Loader					

Table 5.3: End-to-end comparison. Method 2 is the bar PADS-Loader must clear; method 4 is the ceiling. **[TO MEASURE: populate]**

Two comparisons carry the argument. Against method 2 — PADS-Loader must beat a competently hand-tuned loader, not merely the naive default, or the contribution is a tuning guide rather than a system. Against method 4 — if manually pre-staging the whole corpus matches PADS-Loader, then adaptivity bought nothing *for this dataset*, and the honest conclusion is that the contribution reduces to “the dataset fits in scratch”. The defence in that case is not that the result is otherwise, but that PADS-Loader reaches the ceiling without being told scratch is the answer, and continues to function when the corpus does not fit — which Chapter 6 probes directly by constraining scratch capacity. **[TO MEASURE: narrative]**

5.4 RQ4: Can prefetch depth be auto-tuned?

The claim is falsifiable in a specific way: the auto-tuned depth should land at or near the best fixed depth *without having been told it*. If the swept optimum is materially better than the derived depth, the derivation in Section 4 is mis-specified and should be revised rather than defended.

[TO MEASURE: narrative]

Prefetch depth d	Throughput (samples/s)	Peak scratch (GB)
1 (fixed)		
2 (fixed)		
4 (fixed)		
8 (fixed)		
PADS-Loader (auto-tuned, $d =$ [TO MEASURE:])		

Table 5.4: Fixed prefetch depths against the auto-tuned depth. **[TO MEASURE: populate]**

5.5 Profiling overhead

The warm-up window is a cost, and reporting throughput gains without it would overstate the result.

	Absolute (s)	% of total run
Warm-up profiling window		
Archive construction (one-off, offline)		
Net benefit after overhead		

Table 5.5: Cost of adaptation. **[TO MEASURE: populate]**

5.6 Accuracy is preserved

PADS-Loader is only useful if it accelerates the pipeline without degrading segmentation quality. The relevant threshold is stated in advance: any IoU deviation exceeding the ± 0.5 percentage point standard deviation of the reference results (Table 3.1) is a regression to be explained, not noise.

Method	IoU (per-image, all tiles)	Δ vs. baseline	Within tolerance?
1. Baseline		—	—
3. WebDataset			
5. PADS-Loader			

Table 5.6: Segmentation quality under each data path. Reduction convention as Section A.4. **[TO MEASURE: populate]**

One mechanism deserves specific attention rather than a summary statistic. Shard-based reading approximates exact random shuffling by shuffling shard order and maintaining a sample buffer; reduced shuffling randomness is a plausible route by which a throughput optimisation could quietly degrade the model. The comparison in Table 5.6 is therefore not merely a sanity check but a test of that specific hypothesis, and Section 6.5 varies the buffer size to probe it.

[TO MEASURE: narrative]

Chapter 6

Ablation Study

Chapter 5 evaluates PADS-Loader as a whole. This chapter isolates the contribution of its individual mechanisms and probes the conditions under which the design’s assumptions fail.

6.1 Component ablation

PADS-Loader combines three mechanisms: sharding, staging, and adaptive prefetch depth. Table 6.1 enables each in turn.

Shard	Stage	Adaptive d	Throughput	GPU util. (%)	IoU
✓					
	✓				
✓	✓				
✓	✓	✓			

Table 6.1: Component ablation. **[TO MEASURE: populate]**

6.2 Scratch capacity: the regime the design targets

The Bing 1k corpus fits comfortably in node-local scratch, which is the least interesting case for an adaptive stager: a fixed policy of “copy everything up front” suffices. The design is aimed at the regime where it does not fit, and that regime is reached here by synthetically constraining the scratch budget.

Scratch budget (% of corpus)	Derived depth d	Throughput	Staging hit rate
100%			
50%			
25%			
10%			
0% (staging unavailable)	—		—

Table 6.2: Behaviour as scratch capacity is constrained. Method 4 of Section 4.4 is unavailable below 100%; PADS-Loader should degrade gracefully rather than fail. **[TO MEASURE: populate]**

This is the experiment that distinguishes a contribution from an observation. If PADS-Loader matches full pre-staging at 100% and retains most of its advantage at 25%, the adaptivity is doing work. If it collapses to baseline as soon as the corpus does not fit, the honest reading is that the method is a convenient wrapper around “copy the dataset to scratch”.

6.3 Input size and the CORONA variant

Section 3.2 argued that Bing 1k is the least favourable variant for a staging argument, since larger tiles increase bytes per sample and therefore the compute-to-I/O ratio shifts. That argument is tested rather than assumed.

Variant	Bytes/sample	Baseline T_{data}	PADS-Loader speedup
Bing 1k			
Bing 2k			
Bing + CORONA 1k			

Table 6.3: Effect of input configuration. **[TO MEASURE: populate]**

The CORONA variant is of particular interest because it doubles the number of files read per sample — an image, a mask, and a second co-registered image — and therefore stresses metadata cost specifically rather than bandwidth.

6.4 Classifier threshold sensitivity

The classification rules of Section 4 depend on two thresholds, γ_s and γ_d , set to 1.25 and 1.5 respectively. Values chosen by the designer and never varied are a standing weakness in any adaptive system, and the sensitivity is reported here.

γ_d	Policy selected	Throughput	Correct decision?
1.0			
1.25			
1.5 (default)			
2.0			

Table 6.4: Sensitivity of the policy decision to the classification threshold. **[TO MEASURE: populate]**

The relevant question is not which threshold maximises throughput on this workload — that would be tuning on the test set — but how wide the band is over which the decision is unchanged. A decision that flips under a 10% perturbation of γ_d is not robust.

6.5 Shuffling randomness

Shard-based reading trades exact random access for sequential throughput, and approximates shuffling with a sample buffer. This ablation varies the buffer size to establish where, if anywhere, reduced shuffling randomness begins to affect segmentation quality.

Shuffle buffer (samples)	Throughput	IoU	Δ vs. exact shuffle
Exact (loose)			—
4096			
1024			
256			
0 (shard order only)			

Table 6.5: Trade-off between shuffling fidelity and throughput. **[TO MEASURE: populate]**

This is the mechanism by which a throughput optimisation is most likely to degrade the model silently, and it is therefore the ablation most worth reporting even if the result is null.

Chapter 7

Conclusions

This thesis asked whether a profiler-adaptive **DataLoader** that combines bottleneck classification, tar-shard staging and predictive prefetching can improve GPU utilisation and tile throughput for large-scale satellite-image segmentation on HPC systems.

7.1 Summary of contributions

The workload is I/O-bound, and this is measurable. Chapter 3 characterised the segmentation corpus not by its class balance — the conventional description — but by its file layout and access pattern. The Bing 1k configuration comprises 5,025 image and 5,025 mask files stored individually on a shared parallel filesystem, giving a per-sample cost dominated by metadata operations rather than by bytes transferred. Baseline profiling showed a CPU-to-GPU time ratio across all three architectures large enough that the data path is a plausible constraint, and showed that changing two loader flags moves MA-Net IoU by five points and DeepLabV3+ by seven — a swing comparable to the entire architectural search of the reference study.

PADS-Loader. Chapter 4 presented a drop-in loader that profiles its own behaviour over a bounded warm-up window, classifies the dominant bottleneck by an inspectable rule set over measured timings, selects among loose, shard and stage policies, and derives its prefetch depth from measured staging and consumption times subject to a scratch-capacity constraint. The design rests on a property of supervised training that is usually left unused: once the epoch permutation is drawn, future file accesses are known exactly, so prefetching is exact rather than speculative.

An evaluation that isolates mechanisms. [TO MEASURE: headline result: throughput and GPU-utilisation improvement over the tuned baseline, and position relative to the full pre-staging ceiling]

A reproduction, and the defects it surfaced. Appendix A reproduced the reference configuration and established the accuracy floor against which any pipeline change is judged. Appendix B documents the defects encountered, two of which are consequential beyond this thesis: the staging policy was applied to loader construction but not to dataset construction, so the selected policy never reached the data path; and no archive existed in a layout the shard reader could open. Together these mean that staging results collected before these fixes describe policy *decisions* rather than executions. That finding is reported here in full because it invalidates prior measurements rather than merely improving on them.

7.2 Answers to the research questions

RQ1 — When does the loader become the bottleneck? **[TO MEASURE:** answer from Section 5.1, including whether the predicted ordering by T_{gpu} held]

RQ2 — Do tar shards reduce many-small-file overhead? **[TO MEASURE:** answer from filesystem operation counts, and whether the reduction translated into wall-clock benefit]

RQ3 — Does predictive staging outperform fixed streaming? **[TO MEASURE:** answer from 5 vs 3, positioned against the method-4 ceiling]

RQ4 — Can prefetch depth be auto-tuned? **[TO MEASURE:** answer from the depth sweep: did the derived depth land at the swept optimum?]

7.3 What this work does not claim

Three limits are worth stating plainly, because the value of the contribution depends on where its boundary lies.

First, the result is specific to a corpus of many small files on a contended shared filesystem. A workload whose data already resides on fast node-local storage, or whose samples are large enough that per-file overhead amortises, has no bottleneck for PADS-Loader to remove; the loader is designed to detect exactly this and degrade to the ordinary `DataLoader`, and reporting that it does so is part of the claim rather than an admission against it.

Second, PADS-Loader optimises the data path and does not improve the model. Segmentation quality is treated throughout as a constraint to be preserved rather than an objective to be advanced, and any IoU movement beyond the reference standard deviation is treated as a regression to be explained.

Third, the accuracy figures reported in this thesis are drawn from the cluster environment. Reproduction work on a memory-constrained workstation attained substantially lower IoU under an otherwise faithful configuration, traced to the batch-size reduction forced by accelerator memory and its effect on batch-normalisation statistics (Appendix A). That environment was adequate for instrumentation and defect analysis and is not a basis for accuracy claims.

7.4 Closing remarks

The broader observation this work supports is methodological. Effort in deep learning for remote sensing is conventionally directed at the model, and the reference study for this workload found that architectural choice moved IoU by only a few percentage points — plausibly within the noise of random augmentation. Two loader flags moved it by five to seven. When the headroom in a training pipeline is no longer in the architecture, the question of where the accelerator’s time actually goes stops being an implementation detail and becomes the research question.

Chapter 8

Limitations and Future Work

8.1 Limitations

A single filesystem and a single cluster. All primary measurements were taken on GPFS on the Iris cluster. The many-small-file pathology is general to parallel filesystems, but its magnitude is not: Lustre, BeeGFS and object stores differ in metadata architecture, and the threshold at which sharding becomes worthwhile will differ accordingly. The classification rules are expressed as ratios rather than absolute times specifically so that they might transfer, but that transfer is asserted rather than demonstrated.

Contention is uncontrolled. Read latency on a shared filesystem depends on concurrent jobs. Repeats and reporting of variance mitigate this, but do not eliminate it, and a measurement taken on a quiet cluster is not the measurement a user will experience on a busy one. An adversarial evaluation — deliberately co-scheduling I/O-heavy jobs — would strengthen the claim considerably and was not undertaken here.

One corpus. The design generalises to any workload with a fixed epoch permutation over many small files, but it has been evaluated on one dataset of approximately 1 GB. A corpus one or two orders of magnitude larger — where staging cannot be a full copy under any policy — is where the adaptivity should matter most and where it remains untested outside the synthetic capacity constraint of Section 6.2.

Single-node scope. PADS-Loader stages to node-local scratch for a single training process. Multi-node distributed training raises questions the present design does not address: whether shards should be partitioned across ranks or replicated, how staging interacts with a distributed sampler, and whether one rank’s staging traffic degrades another’s read latency.

The classifier is a rule set. Bottleneck classification uses a small number of hand-chosen thresholds. This was a deliberate choice — an inspectable rule set is auditable and its failures are diagnosable, which a learned policy is not — but it means the thresholds are the designer’s judgement. Section 6.4 reports their sensitivity rather than removing the concern.

8.2 Future work

Decode offload. The classifier can already detect the case in which T_{decode} dominates, but its only response is to raise the worker count. GPU-side decode [14] or a cache of

decoded, pre-augmented tensors would give it a substantive action in that regime. The trade-off is memory footprint against decode cost, and the profiler is already measuring the quantities needed to make that choice.

Cross-job coordination. On a shared cluster, several jobs frequently train on the same corpus. Mohan et al. [12] show substantial gains from coordinating caching and eliminating redundant preprocessing across concurrent jobs. Because PADS-Loader already materialises shards on node-local scratch, extending it to publish and reuse staged shards across co-scheduled jobs is a natural next step.

Online rather than one-shot adaptation. The current design profiles once, at the start of the run, and commits. Since contention varies over the lifetime of a job, periodic re-profiling with hysteresis — re-evaluating the policy every N epochs, and switching only on a sustained change — would address the non-stationarity that motivated adaptivity in the first place. This is arguably the most direct extension of the present work.

Revisiting the accuracy question. Two findings from the reproduction bear on segmentation quality rather than throughput and were not pursued here. The first is that per-image IoU is strongly monotonic in the fraction of the crop occupied by the site (Table A.2), with tiles below 2% site area scoring near zero; a size-aware evaluation, or a size-stratified report, would give a more informative picture than a single aggregate. The second is that batch-normalisation statistics degrade badly at the batch sizes imposed by consumer accelerators, which suggests that a normalisation scheme less sensitive to batch size [23] would make this workload substantially more reproducible outside a cluster. Neither is a data-staging question, but both affect who can reproduce this work.

Beyond archaeology. The workload characteristics that motivate PADS-Loader — many small paired files, a fixed permutation, a corpus that may or may not fit node-local scratch — are common across remote sensing, medical imaging and microscopy. Evaluating on a second domain would establish whether the contribution is about this dataset or about a class of pipelines.

Bibliography

- [1] A. Aizman, G. Maltby, and T. Breuel. High performance I/O for large scale deep learning. In *Proc. IEEE Intl. Conf. on Big Data*, 2019.
- [2] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin, and A. A. Kalinin. Albumentations: Fast and flexible image augmentations. *Information*, 11(2):125, 2020.
- [3] L. Casini, N. Marchetti, A. Montanucci, V. Orrù, and M. Roccetti. A human–AI collaboration workflow for archaeological sites detection. *Scientific Reports*, 13:8699, 2023.
- [4] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam. Encoder-decoder with atrous separable convolution for semantic image segmentation. In *Proc. European Conf. on Computer Vision (ECCV)*, 2018.
- [5] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A large-scale hierarchical image database. In *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, pages 248–255, 2009.
- [6] T. Fan, G. Wang, Y. Li, and H. Wang. MA-Net: A multi-scale attention network for liver and tumor segmentation. *IEEE Access*, 8:179656–179665, 2020.
- [7] M. Fiorucci, W. B. V. van der Vaart, P. Soleni, B. L. Saux, and A. Traviglia. Deep learning for archaeological object detection on LiDAR: New evaluation measures and insights. *Remote Sensing*, 14(7):1694, 2022.
- [8] A. Guyot, M. Lennon, T. Lorho, and L. Hubert-Moy. Combined detection and segmentation of archeological structures from LiDAR data using a deep learning approach. *J. Computer Applications in Archaeology*, 4(1), 2021.
- [9] P. Iakubovskii. Segmentation models PyTorch, 2019. GitHub repository.
- [10] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proc. Intl. Conf. on Machine Learning (ICML)*, 2015.
- [11] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár. Focal loss for dense object detection. In *Proc. IEEE Intl. Conf. on Computer Vision (ICCV)*, 2017.
- [12] J. Mohan, A. Phanishayee, A. Raniwala, and V. Chidambaram. Analyzing and mitigating data stalls in DNN training. *Proc. VLDB Endowment*, 14(5):771–784, 2021.

- [13] D. G. Murray, J. Simsa, A. Klimovic, and I. Indyk. tf.data: A machine learning data processing framework. *Proc. VLDB Endowment*, 14(12):2945–2958, 2021.
- [14] NVIDIA Corporation. NVIDIA DALI: Data loading library. Software library.
- [15] H. A. Orengo, F. C. Conesa, A. Garcia-Molsosa, et al. Automated detection of archaeological mounds using machine-learning classification of multisensor and multitemporal satellite data. *Proc. National Academy of Sciences*, 117:18240–18250, 2020.
- [16] A. Paszke, S. Gross, F. Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [17] O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Proc. Intl. Conf. on Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241, 2015.
- [18] M. Tan and Q. V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proc. Intl. Conf. on Machine Learning (ICML)*, 2019.
- [19] Ø. D. Trier, D. C. Cowley, and A. U. Waldeland. Using deep neural networks on airborne laser scanning data: Results from a case study of semi-automatic mapping of archaeological topography on Arran, Scotland. *Archaeological Prospection*, 26:165–175, 2019.
- [20] W. B. V. van der Vaart and K. Lambers. Applying automated object detection in archaeological practice: A case study from the southern Netherlands. *Archaeological Prospection*, 29:15–31, 2022.
- [21] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [22] T. J. Wilkinson. *Archaeological Landscapes of the Near East*. University of Arizona Press, 2003.
- [23] Y. Wu and K. He. Group normalization. In *Proc. European Conf. on Computer Vision (ECCV)*, 2018.
- [24] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo. SegFormer: Simple and efficient design for semantic segmentation with transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

Appendix A

Reproduction of the Reference Configuration

This appendix documents an independent reproduction of the reference configuration of Casini et al. [3], undertaken on a single-GPU workstation before the cluster experiments. It is included for two reasons. First, the accuracy floor asserted in Section 3.2 is only meaningful if it has been established rather than assumed. Second, the reproduction surfaced a set of pipeline defects (Appendix B) and one instructive environmental sensitivity, both of which bear on how the primary results should be read.

A.1 Environment

Reproduction was performed on an NVIDIA RTX 3050 Laptop GPU (4 GB VRAM, driver 566.07), 8 logical CPU cores and 15.7 GB system RAM, running PyTorch 2.13.0+cu126 with CUDA 12.6 and Lightning 2.6.5 under Python 3.12. The corpus was the Bing 1k configuration: 4,734 site tiles and 1,155 empty tiles, 5,889 image/mask pairs totalling 957 MB, split 4,712 train / 588 validation / 589 test under seed 1234.

The site count is consistent with the unfiltered variant of the published corpus: $4,934 - 200 = 4,734$, i.e. the top-200-by-area removal applied but not the 684-site minimum-size filter. The applicable reference figure is therefore Model 1 of Table 3.1, 74.17 ± 0.38 .

A.2 Faithfulness of the port

The reference implementation was ported directly rather than reimplemented. The following were held identical to the published notebook: MA-Net with an EfficientNet-B3 ImageNet-pretrained encoder; `FocalLoss(mode=BINARY)` with α unset; `Adam` at 10^{-4} ; `precision=32`; 20 epochs; a `RandomCrop(512)` followed by `Resize(256)` for both training and validation; augmentation by single-axis flip, 90-degree rotation and brightness/contrast jitter, all at $p = 0.25$; standardisation of raw 0–255 input by ImageNet statistics without prior division by 255; evaluation by `smp.metrics` with `reduction="macro-imagewise"`; final-epoch weights rather than best-validation weights; and ten test passes with independent random crops.

Two adaptations were unavoidable. The published notebook targets PyTorch Lightning 1.x, whose `training_epoch_end` and `validation_epoch_end` hooks were removed in 2.x; per-epoch aggregation was therefore reimplemented explicitly. The `A.Flip` transform has since been removed from `augmentations`; its semantics — selecting one of {vertical, horizontal, both} uniformly, rather than applying two independent flips — were reproduced explicitly and verified empirically over 4,000 samples, yielding 0.752 no-flip and 0.089 / 0.084 / 0.075 for the three flip modes against an expected 0.750 / 0.083 each.

A.3 Result and the batch-size constraint

Under this configuration the reproduction attained an IoU of $\mathbf{0.4470 \pm 0.0059}$ under the reference reduction, against the published 0.7417 ± 0.0038 .

The discrepancy is attributable to accelerator memory. The reference configuration trains at batch size 32. Measured peak allocation for MA-Net/EfficientNet-B3 at 256×256 in fp32 is 2.44 GB at batch 8, 4.65 GB at batch 16, and out-of-memory at batch 32 on a 4 GB card; at half precision, batch 32 requires 4.94 GB and therefore also does not fit. Batch 32 is thus unreachable on this hardware at any precision. The reproduction used batch 8 with gradient accumulation $\times 4$, which preserves the effective batch — and hence the gradient signal — but leaves batch normalisation estimating its statistics from 8 samples per forward pass rather than 32.

EfficientNet is batch-normalisation dense, and the degradation of BN statistics at small batch is a well-characterised effect; it is the motivating problem for Group Normalization [23, 10]. That this is the operative cause rather than a defect in the port is corroborated by the cluster measurements of Section 3.4.4, where the same code at batch 32 on a V100 attains 68–73% IoU, consistent with the published figure.

Consequence for this thesis. All accuracy figures reported in the main text are drawn from the cluster. The workstation environment is used for development, defect analysis and instrumentation work, for which it is adequate, and not for any claim about segmentation quality.

A.4 Metric reduction

The reduction convention accounts for a material part of the difference between naively reported and reference-comparable numbers. Measured on the same trained checkpoint (Table A.1):

Reduction	IoU
Globally pooled, site-bearing tiles only	0.4131
Per-image mean, site-bearing tiles only	0.3364
Per-image mean, all tiles (reference convention)	0.4504

Table A.1: Effect of the aggregation convention on reported IoU for an identical model. The reference convention counts a correctly-predicted empty tile as a perfect match.

Two hypotheses about the metric were tested and rejected. Excluding the empty tiles entirely changes pooled IoU by 0.001, because they contribute only 0.8% of false positives — the model classifies 102 of 104 empty tiles perfectly. Sweeping the decision threshold over $\{0.2, \dots, 0.7\}$ confirms 0.5 is optimal, so the reported figure is not an artefact of a poorly chosen operating point.

A.5 Error structure

The aggregate figure conceals a strongly bimodal error distribution. On the validation split, per-image IoU has a 90th percentile of 0.785 — at reference level — while 35.3% of tiles score below 0.1. Performance is strongly monotonic in the fraction of the crop occupied by the site (Table A.2).

Site area in crop	Tiles	Mean IoU	Fraction < 0.1
0.5–2%	25	0.024	0.92
2–5%	83	0.179	0.57
5–12%	137	0.285	0.37
12–30%	151	0.440	0.19
30–100%	86	0.488	0.23

Table A.2: Per-image IoU by the fraction of the cropped tile occupied by the annotated site.

This is consistent with the original authors’ rationale for excluding sites below 1,000 m²: on small objects, a few pixels of boundary error destroys the ratio, and such annotations frequently denote sites of imprecise location rather than precise small mounds. It also explains the +3.9 point gain the reference reports from filtering (Model 1 → Model 2).

Appendix B

Pipeline Defects Identified During Reproduction

Porting the training code to a second environment surfaced a number of defects in the experimental pipeline. They are documented here because several are silent — they degrade results or invalidate measurements without raising an error — and because two of them invalidate earlier data-staging measurements and therefore bear directly on the claims this thesis is able to make.

B.1 Defects affecting the staging mechanism

B.1.1 Policy selected after the datasets it selects were constructed

The staging policy was applied only to `DataLoader` construction, not to dataset construction. In the original control flow, the three datasets were built before the profiler ran:

```
1 policy = base_config.get("data_policy", "loose")    # key absent -> "loose"
2
3 train_dataset = create_dataset(..., policy=policy) # always the loose
4 reader
5 ...
6 base_config["data_policy"] = policy_summary["policy"] # profiler decides
7 , too late
8 train_loader = build_dataloader(train_dataset, policy=base_config["
9 data_policy"])
```

The datasets were never rebuilt after the profiler had spoken, and the selected policy reached only `build_dataloader`, whose entire use of it is to choose `num_workers` and `prefetch_factor`. The shard and stage reader classes were therefore unreachable code, and every run — irrespective of the policy the profiler reported — read loose files.

The consequence is that any previously reported staging result describes a policy *decision* rather than a policy *execution*. The fix reorders policy selection ahead of dataset construction, with the profiler probing an explicitly-loose dataset and the working datasets built from its verdict.

B.1.2 No archive the shard reader could open

The shard reader addresses members by original filename within archives named `originals.tar` and `negs.tar`. The repository’s shard generator produces WebDataset-format shards (`shard-000000.tar`, members keyed `train_000123.jpg`), which that reader can neither open nor address; and no archives of either form existed on disk. The shard branch would therefore have fallen through to its loose fallback even had it been reachable.

Archives in the required layout are now produced by a dedicated builder, which verifies its output by comparing decoded samples against the loose path. They are written uncompressed for the reason given in Section 4: the payload is already-compressed JPEG and PNG, so compression yields negligible size reduction while forcing decompression from the stream start on every random access.

B.1.3 Staging silently degraded to a plain read

The staging dataset copies samples to a scratch directory but swallows `IOError` and `OSError` during the copy. The scratch directory was never created, so every copy failed silently and the policy read directly from the source path — appearing selected and active while having no effect.

B.2 Defects affecting segmentation accuracy

Defect	Description and evidence
Focal loss α unset in one variant	<code>smp.losses.FocalLoss</code> defaults to <code>alpha=None</code> , applying no class weighting on a target with a median 7% positive-pixel rate.
Validation on a random crop	Scoring a different random patch of each tile every epoch injects variance into checkpoint selection and early stopping. Note that the reference <i>deliberately</i> random-crops at test time and averages ten passes; the defect is using it as an unaveraged per-epoch selection signal.
Configured loss silently ignored	One model hardcoded <code>BCEWithLogitsLoss</code> while both the configuration and the training log reported <code>focal</code> . Correcting it moved epoch-1 IoU from 0.299 to 0.362 and reversed a declining validation trend. Unknown loss names now raise rather than falling through.
fp16 divergence on EfficientNet	Under <code>16-mixed</code> , training loss became non-finite at epoch 4 and all metrics collapsed to zero, while early stopping continued for its full 15-epoch patience on a dead model; the run reported a “best” score that was merely the last value before the blowup. ResNet-50 was unaffected across 20 epochs. Isolating the loss under <code>autocast</code> showed gradients bit-identical to fp32, locating the overflow in the encoder — consistent with EfficientNet’s SiLU and squeeze-excite blocks exceeding fp16’s exponent range. Resolved by <code>bf16-mixed</code> ; a guard now aborts at the epoch a loss becomes non-finite.

Table B.1: Accuracy-affecting defects.

B.3 Defects affecting reproducibility and operations

- **Metric-name path separator.** Checkpoint filenames interpolate the monitored metric name `valid/iou` verbatim. On filesystems treating `/` as a path separator this creates a directory rather than a filename, and `save_top_k` then prunes the checkpoint while leaving an orphaned directory per improved epoch.
- **Profiler perturbs the training RNG.** The warm-up pass iterates the *training* dataset, whose transforms are stochastic, advancing the global random state before training begins. A profiled run is therefore not seed-comparable with an unprofiled one at identical configuration. Any comparison between them must either restore RNG state after profiling or treat the difference as within run-to-run variance, which was measured at approximately ± 0.02 IoU.
- **Study persistence.** Hyperparameter studies were not persisted, so an interrupted sweep restarted from trial zero and reused checkpoint directories.

B.4 Implication for prior results

Sections [B.1.1](#) and the missing archives are the consequential ones. Because the selected policy never reached the data path, and because the shard policy had no readable data in any case, staging results collected before these fixes report decisions rather than executions. They are not recoverable by reanalysis and the corresponding experiments require repetition. The defects are properties of the code rather than of any particular machine, and therefore applied equally to cluster runs.